



OS-Project1

주제

CFS scheduling 구현

작성자

전예원 / 컴퓨터소프트웨어학부 / 2023096608 / 3학년 / 12913

목차

1. Design
2. Implementation
3. Results
4. Troubleshooting

Design

방향성

CFS는 lowest virtual runtime를 가진 running process를 골라 execute하는 scheduling algorithm이다. 이를 위해 red-black tree를 사용한다.

- Virtual Runtime은 각 process마다 가진다고 했으니, process 구조체 안에 정의되어야 한다.
- priority인 nice value에 따라 virtual runtime 값 증가 속도가 다르다. 따라서 이는 process마다 nice 값을 가지고 virtual runtime 증가 속도를 결정해줘야하니, process 구조체에 역시 정의되어야 한다.
- 또한 runnable 상태일 때, red-black tree에 insert하고 이후 delete할 수 있도록 red-black tree의 node도 정의되어야 한다.
- 그리고 running 상태 일 때 CPU 사용 시간을 구해 virtual runtime을 계산하기 위하여 running 상태가 된 시점을 기록하는 변수도 선언해야 한다고 판단했다.

핵심 아이디어

- runnable process는 모두 red-black tree에 저장한다.

- 이후 scheduler는 tree의 최소 노드(root)를 확인하여 가장 작은 virtual runtime을 가진 process를 선택한다.
- vruntime은 yield되거나, sleep, 종료할 때 실제 실행시간을 측정하고 가중치인 nice 값을 곱하여 갱신한다.
- 만약, 자식 process가 새로 생성된다면 부모의 nice와 virtual runtime을 상속 받도록 한다.
- sleep 상태에서 깨어나는 process는 runnable 또는 running 프로세스들의 최소 virtual runtime 기준으로 자신의 vruntime을 재설정하도록 한다.

Lock 관련 설계

해당 scheduler를 multiprocessor 환경에서 deadlock이 걸리지 않게 구현하기 위하여 3가지 lock이 필요하다고 판단하였다.

- 먼저, process 별 상태를 보호하는 lock인 p→lock은 기본적으로 필요하다.
- 그리고 multiprocessor 환경에서는 특히 red-black tree의 삽입과 삭제도 lock으로 보호하는 것이 필수라고 판단하였다.
- ticks는 공유 변수이며, CPU burst time을 정확하게 계산하기 위하여 tickslock도 도입하기로 결정하였다.

⇒ 특히, process 상태 변경과 tree 조작이 동시에 일어나는 구간에서는 race condition이 발생할 수 있으므로 상태 재확인 필요하다.

Implementation

1. `proc.h` 에서 process 구조체 수정

```
// Per-process state
struct proc {
    struct spinlock lock;

    ...

    //CFS를 위해 추가하는 변수
    int nice;
    uint64 vruntime;
    struct rb_node *rbNode;
    uint64 run_start_tick; //running 상태 시작 시간
};
```

- nice는 vruntime 축적 속도를 결정하며, set_nice system call로 결정되므로 process마다 정의했다.
- vruntime은 CFS scheduling의 핵심 field이며, process마다 가져야하므로 정의했다.

- rbNode는 red-black tree에서 해당 process가 나타내는 node를 저장하기 위해 도입했다.
- run_start_tick은 process가 CPU를 잡은 시간을 계산하기 위해 추가적으로 도입했다.

2. **set_nice** system call 등록

proc.c

```
//set_nice system call
int set_nice(int nice){
    struct proc* p = myproc();

    if(nice<-3 || nice>2) return 1;

    acquire(&p->lock);
    p->nice = nice;
    release(&p->lock);

    return 0;
}
```

- 현재 실행중인 process의 nice 값을 설정하기 위한 function이다.
- parameter로 들어온 nice 값이 -3 미만이거나 2 초과이면 잘못된 nice 값이므로 return 1 해준다.
- nice 값을 설정하는 부분은 race condition이므로 process lock을 걸어 보호해주었다.
- 이후 return 0를 통해 값이 잘 설정됨을 알 수 있다.

sysproc.c

```
//set_nice에 대한 wrapper
uint64
sys_set_nice(void){
    int nice;

    argint(0, &nice);

    return set_nice(nice);
}
```

user program에서 trapframe을 통해 넘어온 nice 인자를 처리하고 set_nice system call을 호출하기 위한 wrapper function을 정의하였다.

syscall.h

```
#define SYS_set_nice 22
```

22번으로 system call 번호를 지정해주었다.

syscall.c

```
extern uint64 sys_set_nice(void);

static uint64 (*syscalls[])(void) = {
    ...
    [SYS_mkdir]    sys_mkdir,
    [SYS_close]    sys_close,
    [SYS_set_nice] sys_set_nice,
};
```

syscall.c에서 system call로 등록해주었다.

user.h

```
int set_nice(int nice);
```

user mode에서 system call을 인식하고 호출할 수 있도록 header file에 정의하였다.

usys.pl

```
entry("set_nice");
```

system call을 호출하는 assembly code를 생성하기 위해 usys.pl에도 정의하였다.

3. CFS scheduling 상세 구현

3-1. 필요한 전역 변수 선언, 함수 선언 및 구현

```
...
#include rbtree.h
...
#include <stddef.h>

//CFS를 위한 function 및 변수 추가
struct rb_tree cfs_runnable_tree;
struct spinlock cfs_lock; //다중 CPU에서 문제
```

- 먼저 rbtree를 사용하기 위해 rbtree header를 include하였다.

- NULL 타입을 사용하기 위해 `stddef.h`도 추가로 도입하였다.
- runnable 상태의 process를 담고 있을 red-black tree를 `cfs_runnable_tree`로 선언하였다.
- 그리고 red-black tree의 race condition을 해결하기 위해 `cfs_lock`을 도입하였다.

```
static uint64
niceWeight(int nice)
{
    switch(nice){
        case -3: return 1;
        case -2: return 2;
        case -1: return 4;
        case 0:  return 8;
        case 1:  return 16;
        case 2:  return 32;
        default: return 8;
    }
}
```

nice 값에 따른 vruntime의 증가 비율은 nice 값이 1 증가할수록 vruntime의 amount가 double이 되므로, nice 값 -3을 기준 1로 잡은 뒤 각 nice에 맞는 weight를 return하게 설계해주었다.

- 소수와 분수로 나타내면 계산이 복잡해지기 때문에 -3을 기준으로 잡았다.
- 제일 처음 process는 nice 값이 0이어야 하므로 default 값을 8로 해주었다.

```
static void
update_vruntime(struct proc *p){
    uint64 now, burstTime;

    acquire(&tickslock);
    now = ticks;
    release(&tickslock);

    if(now > p->run_start_tick){
        burstTime = now - p->run_start_tick;
        p->vruntime += burstTime * niceWeight(p->nice);
    }

    p->run_start_tick = now;
}
```

해당 함수는 running 상태에서 yield, kexit, sleep으로 더이상 running 상태가 아닐 때, 해당 process의 state를 바꾸며 running 상태일 때의 burst time과 nice 가중치를 이용하여 vruntime을

계산하여 해당 process의 vruntime을 update 해준다.

- ticks를 이용하여 시간 차이를 계산하는데, scheduler나 해당 함수에서 ticks 값을 읽을 때 다른 CPU나 interrupt handler가 개입하여 해당 값을 바꿀 수도 있기 때문에 tickslock으로 보호하였다.
- 이후 burstTime으로 running 상태로 바뀐 후부터 실제 CPU 사용 시간을 구해준다.
- 그리고 vruntime은 이 burstTime에 nice 가중치를 곱하여 계산되며, 해당 계산값을 vruntime에 축적해주면 total CPU burst time을 알 수 있다.
- 마지막으로 process의 run_start_tick을 now로 바꿔주는데, 이건 CPU 사용 끝마친 시간을 기록하는 것이며, 이후 scheduler에서는 다시 run_start_tick을 갱신하기 때문에 확인용으로 첨가하였다.

```
//wakeup시 update 규칙 추가
static uint64
wakeup_vruntime_setting(void){
    struct proc* p;
    struct rb_node *node;

    //runnable중 최소 vruntime
    acquire(&cfs_lock);
    node = rb_first(cfs_runnable_tree.root);
    if(node != NULL){
        uint64 targetVruntime = ((struct proc*)node->data)->vruntime;
        release(&cfs_lock);
        return targetVruntime;
    }
    release(&cfs_lock);

    //runnable이 없다면, running중 최소 vruntime(다중 CPU에서는 running
    여러개..)
    uint64 minVruntime = (uint64)-1;
    int found = 0;

    for(p = proc; p<&proc[NPROC]; p++){
        acquire(&p->lock);
        if(p->state == RUNNING){
            if(!found || p->vruntime < minVruntime){
                minVruntime = p->vruntime;
                found = 1;
            }
        }
        release(&p->lock);
    }
}
```

```

}

//둘다 없으면 0
if(found) return minVruntime;
return 0;
}

```

해당 함수는 block된 process가 wakeup 했을 때 CPU scheduling의 대상이 되는데, 이 때 깨어난 process가 너무 유리하지 않도록 vruntime을 정해준다.

- 먼저, red-black tree에 있는 runnable process 중 최소 vruntime을 가지는 process를 꺼내는데, 이는 red-black tree의 root만 확인해도 된다. 이때, red-black tree를 read하고 check하는 부분은 cfs_lock을 걸어 보호해준다.
- cfs_runnable_tree에서 root인 node를 꺼낸 뒤, 해당 node가 존재하면 꺼낸 node의 process 정보를 확인해서 해당 vruntime을 return하면 된다.
- 만약 runnable한 process가 하나라도 없다면, running한 process 중 최소 vruntime을 고르는데, 이때 process state가 바뀌면 안되므로 process lock을 걸어준다.
- 먼저, minVruntime과 found 변수를 선언하고, process 배열을 돌면서 RUNNING 상태인 process를 찾아 최초로 RUNNING 상태인 process를 찾거나 해당 vruntime이 현재 minVruntime보다 작다면 minVruntime을 update해준다.
- running한 process를 통해 최소 vrunTime을 찾았으면 해당 값을 return하고, 없으면 0을 return한다.

3-2. proc.c의 각 function별 구현한 부분

```

// initialize the proc table.
void
procinit(void)
{
    struct proc *p;

    initlock(&pid_lock, "nextpid");
    initlock(&wait_lock, "wait_lock");
    initlock(&cfs_lock, "cfs_lock");

    cfs_runnable_tree.root = NULL;

    for(p = proc; p < &proc[NPROC]; p++) {
        initlock(&p->lock, "proc");
        p->state = UNUSED;
    }
}

```

```

    p->kstack = KSTACK((int) (p - proc));

    p->nice = 0;
    p->vruntime = 0;
    p->rbNode = NULL;
    p->run_start_tick = 0;
}
}

```

- 먼저, procinit에서는 process table을 초기화하므로 lock도 함께 초기화해준다.
- 이후, process table을 돌면서 해당 process의 lock과 상태, 그리고 nice, vruntime, rbNode, run_start_tick 등을 초기화해준다.

```

static struct proc*
allocproc(void)
{
    struct proc *p;

    for(p = proc; p < &proc[NPROC]; p++) {
        acquire(&p->lock);
        if(p->state == UNUSED) {
            goto found;
        } else {
            release(&p->lock);
        }
    }
    return 0;

found:
    p->pid = allocpid();
    p->state = USED;

    //CFS 초기화
    p->nice = 0;
    p->vruntime = 0;
    p->rbNode = (struct rb_node *)kalloc();
    if(p->rbNode == NULL) { //제대로 할당됐는지 check!!
        p->state = UNUSED;
        release(&p->lock);
        return 0;
    }
    p->rbNode->key = 0;
}

```

```

p->rbNode->data = (void *)p;
p->rbNode->parent = NULL;
p->rbNode->left = NULL;
p->rbNode->right = NULL;
p->run_start_tick = 0;

// Allocate a trapframe page.
...

return p;
}

```

allocproc에서는 UNUSED한 상태인 경우 USED로 바꿔주고 해당 process에 pid를 부여하는 함수이다.

- 이 때 CFS 관련 선언 변수(nice, vruntime 등)을 default 값으로 초기화해주고, rbNode에는 동적할당을 통해 Node를 생성해놓는다.
- 그리고 해당 Node의 value들은 모두 초기 값으로 setting하면 된다.

```

static void
freeproc(struct proc *p)
{
    if(p->trapframe)
        kfree((void*)p->trapframe);
    p->trapframe = 0;
    if(p->pagetable)
        proc_freepagetable(p->pagetable, p->sz);
    p->pagetable = 0;
    p->sz = 0;
    p->pid = 0;
    p->parent = 0;
    p->name[0] = 0;
    p->chan = 0;
    p->killed = 0;
    p->xstate = 0;
    p->state = UNUSED;

    //free하니까 CFS 관련 초기화
    p->nice = 0;
    p->vruntime = 0;
    if(p->rbNode){
        kfree((void*)p->rbNode);
        p->rbNode = NULL;
    }
}

```

```

    }
    p->run_start_tick = 0;
}

```

freeproc은 process 할당을 해제하는 것이므로, process 구조체 안에 선언된 변수들도 안전하게 초기화해준다.

- 특히, CFS 관련으로 선언한 nice, vruntime, run_start_tick은 0으로 setting하고, rbNode는 allocproc에서 동적 할당했으므로, kfree가 반드시 이루어져야 한다.

```

// Set up first user process.
void
userinit(void)
{
    struct proc *p;

    p = allocproc();
    initproc = p;

    p->cwd = namei("/");

    //첫 user process이니 vruntime=0
    p->nice = 0;
    p->vruntime = 0;
    p->rbNode->key = p->vruntime;
    p->rbNode->data = (void *)p;

    p->state = RUNNABLE;
    acquire(&cfs_lock);
    rb_insert(&cfs_runnable_tree, p->rbNode);
    release(&cfs_lock);

    release(&p->lock);
}

```

userinit은 첫 user process를 할당하는 function이다. 해당 process에 대한 CFS 관련 값을 선언 해서 red-black tree에 넣어주는 코드만 추가해주면 된다.

- 첫 user process는 nice 값이 0이므로 해당 값을 설정해주고 rbNode의 key에 vruntime을 setting하고, rbNode의 data로는 해당 process를 참조하도록 해준다.
- 그리고 cfs_runnable_tree에 넣기전, state를 runnable로 바꾸고 red-black tree에 insert해 준다.

- cfs_runnable_tree에 접근하는 것이 race condition이므로 cfs_lock을 통해 안전하게 insert 한다.

```
int
kfork(void)
{
    int i, pid;
    struct proc *np;
    struct proc *p = myproc();

    // Allocate process.
    if((np = allocproc()) == 0){
        return -1;
    }

    ...

    np->nice = p->nice;
    np->vruntime = p->vruntime;
    np->rbNode->key = np->vruntime;
    np->rbNode->data = (void *)np;
    np->rbNode->parent = NULL;
    np->rbNode->left = NULL;
    np->rbNode->right = NULL;

    pid = np->pid;

    release(&np->lock);

    acquire(&wait_lock);
    np->parent = p;
    release(&wait_lock);

    acquire(&np->lock);
    np->state = RUNNABLE;

    acquire(&cfs_lock);
    rb_insert(&cfs_runnable_tree, np->rbNode);
    release(&cfs_lock);

    release(&np->lock);
}
```

```

    return pid;
}

```

kfork는 parant process가 자식 process를 만들고 memery space를 복사하는 function이다.

- 이 때, 자식 process는 부모의 nice 값과 vruntime을 물려받게 setting하면 된다.
- 그리고 rbNode→data에 자식 process를 넣어주고, parent, left, right는 red-black tree에서의 parent와 left, right이므로 혼동하지 않고 NULL 값으로 넣어주면 된다.
- 그리고 runnable로 state를 바꾼 뒤 cfs_lock을 건 뒤 cfs_runnable_tree에 해당 process의 Node를 insert 해주면 된다.

```

void
kexit(int status)
{
    struct proc *p = myproc();

    ...

    acquire(&wait_lock);

    // Give any children to init.
    reparent(p);

    // Parent might be sleeping in wait().
    wakeup(p->parent);

    acquire(&p->lock);

    update_vruntime(p);
    p->xstate = status;
    p->state = ZOMBIE;

    release(&wait_lock);

    // Jump into the scheduler, never to return.
    sched();
    panic("zombie exit");
}

```

kexit은 해당 process를 exit 한 뒤 zombie 상태로 바뀌고, 부모 process가 wait하길 기다리는 함수이다.

- 해당 function에서는 process가 exit되므로, zombie로 state가 바뀌기 전, CPU burst time을 계산하여 vruntime을 update 해야하므로 update_vruntime 함수를 호출해서 vruntime을 update한다.

```
void
scheduler(void)
{
    struct proc *p;
    struct cpu *c = mycpu();
    struct rb_node* node;

    c->proc = 0;
    for(;;){
        // The most recent process to run may have had interrupts
        // turned off; enable them to avoid a deadlock if all
        // processes are waiting. Then turn them back off
        // to avoid a possible race between an interrupt
        // and wfi.
        intr_on();
        intr_off();

        int found = 0;
        acquire(&cfs_lock);
        node = rb_first(cfs_runnable_tree.root);
        release(&cfs_lock);

        if(node != NULL) {
            p = (struct proc *)node->data;

            acquire(&p->lock);
            acquire(&cfs_lock);

            //runnable이면서 최소 노드가 맞는지 확인(멀티 CPU일때 필수)
            if(p->state == RUNNABLE && rb_first(cfs_runnable_tree.root)
            == p->rbNode) {

                //tree에서 제거
                rb_delete(&cfs_runnable_tree, p->rbNode);
                release(&cfs_lock);

                //CPU 쓰기 시작한 시간 기록
                acquire(&tickslock);
                p->run_start_tick = ticks;
            }
        }
    }
}
```

```

        release(&tickslock);

        // Switch to chosen process. It is the process's job
        // to release its lock and then reacquire it
        // before jumping back to us.
        p->state = RUNNING;
        c->proc = p;
        switch(&c->context, &p->context);

        // Process is done running for now.
        // It should have changed its p->state before coming back.
        c->proc = 0;
        found = 1;
    }else{
        release(&cfs_lock);
    }
    release(&p->lock);
}
if(found == 0) {
    // nothing to run; stop running on this core until an interr
upt.
    asm volatile("wfi");
}
}
}
}

```

scheduler에서는 cfs_runnable_tree에 있는 runnable process 중 최소 vruntime을 선택해서 RUNNING 상태로 바꾸고 CPU 사용시간을 측정해주도록 수정해주었다.

- 먼저 cfs_lock을 통해 cfs_runnable_tree에서 root 값을 꺼내고, 해당 node가 존재하면, node의 data인 process를 꺼내서 p에 담는다
- 이후 process 관련 race condition을 없애기 위해 p → lock을 건뒤, cfs_lock을 걸어서 red-black tree 관련 작업을 수행해주었다.
- lock 걸기 전 멀티 CPU에 의해 process state가 바뀔수도 있으니 lock을 건 다음 해당 node에 대한 RUNNABLE 상태와 root를 다시 한번 확인해주었다.
- 그리고 tree에서 root를 제거한 뒤 cfs_lock을 풀어주고 CPU를 쓰기 시작한 시간을 tickslock을 통해 기록해주었다. 이것 역시 다른 CPU에 의해 ticks가 의도치 않은 값이 들어가지 않도록 설계해준 것이다.
- 그리고 해당 process를 RUNNING 상태로 바꾼 뒤 CPU 사용을 시작한다.

```

// Give up the CPU for one scheduling round.
void

```

```

yield(void)
{
    struct proc *p = myproc();
    acquire(&p->lock);
    //실행 시 vruntime을 nice에 맞게 반영해야함
    update_vruntime(p);

    p->state = RUNNABLE;

    p->rbNode->key = p->vruntime;
    p->rbNode->data = (void *)p;
    acquire(&cfs_lock);
    rb_insert(&cfs_runnable_tree, p->rbNode);
    release(&cfs_lock);

    sched();
    release(&p->lock);
}

```

yield는 CPU를 포기하고 runnable로 돌아갈 때 호출하는 함수이므로 vruntime에 대한 update가 필수적이다.

- 먼저, process lock을 통해 다른 CPU에 의해 process 상태가 변하는 걸 막아준다.
- 이후 update_vruntime을 호출하여 vruntime을 nice 가중치와 CPU burstTime을 이용하여 갱신해준다.
- 그리고 p->state를 runnable로 바꾸고 key를 vruntime을 update하고, data를 해당 process로 설정하여 node 상태를 확실하게 맞춰준다.
- 그리고 cfs_lock을 건 뒤, cfs_runnable_tree에 해당 runnable process를 insert 해주면 된다.

```

void
sleep(void *chan, struct spinlock *lk)
{
    struct proc *p = myproc();

    // Must acquire p->lock in order to
    // change p->state and then call sched.
    // Once we hold p->lock, we can be
    // guaranteed that we won't miss any wakeup
    // (wakeup locks p->lock),
    // so it's okay to release lk.

    acquire(&p->lock); //DOC: sleeplock1

```

```

    release(&lk);

    update_vruntime(p);

    // Go to sleep.
    p->chan = chan;
    p->state = SLEEPING;

    sched();

    // Tidy up.
    p->chan = 0;

    // Reacquire original lock.
    release(&p->lock);
    acquire(&lk);
}

```

sleep은 process가 CPU를 사용하지 않도록 process를 SLEEPING 상태로 만들어주는 것이므로, sleep으로 들어가기 전, vruntime을 update 해야한다. 따라서 update_vruntime(p);를 호출하여 vruntime을 갱신해준다.

```

// Wake up all processes sleeping on channel chan.
// Caller should hold the condition lock.
void
wakeup(void *chan)
{
    struct proc *p;
    uint64 myVruntime;

    for(p = proc; p < &proc[NPROC]; p++) {
        if(p != myproc()){
            acquire(&p->lock);
            if(p->state == SLEEPING && p->chan == chan) {
                release(&p->lock);

                myVruntime = wakeup_vruntime_setting();

                acquire(&p->lock);
                if(p->state == SLEEPING && p->chan == chan){
                    p->state = RUNNABLE;
                    p->vruntime = myVruntime;
                    p->rbNode->key = p->vruntime;
                }
            }
        }
    }
}

```

```

        p->rbNode->data = (void *)p;

        acquire(&cfs_lock);
        rb_insert(&cfs_runnable_tree, p->rbNode);
        release(&cfs_lock);
    }
}
release(&p->lock);
}
}
}

```

wakeup은 sleeping한 process를 깨워서 runnable한 상태로 바꿔주는 function이다.

- 먼저, process table을 순회하며 SLEEPING한 process를 찾은 뒤, p→lock을 잠시 풀어준다.
- 이는 wakeup_vruntime_setting 안의 lock과 충돌하지 않게 하기 위해 해당 방법으로 설계하였다.
- wakeup_vruntime_setting을 통해 얻은 myVruntime을 가진 뒤 p→lock을 다시 걸어 process의 state를 다시 한번 확인해줘야한다. lock이 풀린 사이 다른 CPU가 상태를 바꿀 수도 있기 때문이다.
- 조건을 만족하면 RUNNABLE 상태로 바꿔주고 vruntime을 myVruntime으로 update 해준 뒤 rbNode도 조건에 맞게 수정해준다.
- 그리고 cfs_lock을 걸어 cfs_runnable_tree에 해당 process의 rbNode를 insert 해준다.

```

int
kkill(int pid)
{
    struct proc *p;
    uint64 myVruntime;

    for(p = proc; p < &proc[NPROC]; p++){
        acquire(&p->lock);
        if(p->pid == pid){
            p->killed = 1;
            if(p->state == SLEEPING){
                release(&p->lock);

                myVruntime = wakeup_vruntime_setting();

                acquire(&p->lock);
                if(p->pid == pid && p->state == SLEEPING){
                    // Wake process from sleep().

```

```

        p->state = RUNNABLE;
        p->vruntime = myVruntime;
        p->rbNode->key = p->vruntime;
        p->rbNode->data = (void *)p;

        acquire(&cfs_lock);
        rb_insert(&cfs_runnable_tree, p->rbNode);
        release(&cfs_lock);
    }
    release(&p->lock);
    return 0;
}
release(&p->lock);
return 0;
}
release(&p->lock);
}
return -1;
}

```

kkill은 process를 runnable한 상태로 바꿔준 뒤 CPU scheduler의 선택을 받으면 해당 process를 kill하도록 하기 위해 설계된 function이다. 전체적인 흐름은 wakeup과 유사하다.

- process table을 순회하며 parameter로 들어온 pid를 killed로 setting 해주고 SLEEPING이면 깨워서 runnable하게 만들어준다.
- 이를 위해 wakeup_vruntime_setting()을 불러 myVruntime을 계산해주고 다시 pid와 SLEEPING 조건을 확인해준다. 역시 lock이 풀렸을 때 다른 CPU가 해당 state를 바꿀 수도 있기 때문이다.
- 조건을 만족하면 state를 RUNNABLE로 바꿔주고 vruntime과 rbNode도 알맞게 setting 해준다.
- 마지막으로 cfs_lock을 걸어 cfs_runnable_tree에 해당 node를 넣어주면 된다.

Results

결과를 보기 위한 테스트 케이스는 github에 올라와있는 것을 사용하였다.

make qemu CPUS=1으로 단일 CPU에 대해서 testcase를 실행하고, 멀티 CPU 환경에서는 deadlock 여부만 체크해보았다.

1. testcase1

usertests에 해당 코드를 추가해서 test 하였다.

```

int cfs_test1(char* s){
    printf("\n");
    for (int nice = 2; nice >=
-3; nice--){

        int pid = fork();
        if (pid < 0){
            printf("fork failed
\n");
            exit(1);
        }

        if (pid == 0){
            set_nice(nice);
            for (volatile int i =
0; i < 1000000000; i++){
                printf("pid: %d, nice:
%d\n", getpid(), nice);
                exit(0);
            }
        }

        for (int i = 0; i < 6; i++)
        {
            wait(0);
        }

        return 0;
    }
}

```

```

$ usertests cfs_test1
usertests starting
test cfs_test1:
pid: 10, nice: -3
pid: 9, nice: -2
pid: 8, nice: -1
pid: 7, nice: 0
pid: 6, nice: 1
pid: 5, nice: 2
PASS
PASS ALL TESTS

```

CPUS=1에서 의도한 대로 결과가 잘 나옴을 확인하였다.

2. testcase2

nice 값에 따른 CPU 점유 비율을 보기 위한 test도 실행해보았다.

유의미한 결과를 얻기 위해서는 ticks가 1000 이상이 되어야 함을 확인하여 1000으로 설정하고 test 했다.

```

===== CFS Scheduler Test =====

=== Test 1: Equal nice (all nice=0) ===
Process 1 (nice=0): count = 1265851
Process 2 (nice=0): count = 1254329
Process 3 (nice=0): count = 1260707
Ratio P1/P2: 1.00
Ratio P1/P3: 1.00
Expected: ratios close to 1.00

=== Test 2: Different nice (-3 vs 2) ===
nice=-3 weight=1, nice=2 weight=32
Expected ratio: nice=-3 gets ~32x more CPU
nice=-3 count: 3687874
nice= 2 count: 118421
Ratio (nice=-3 / nice=2): 31.14
Expected: ~32.00

=== Test 3: Adjacent nice (0 vs 1) ===
nice=0 weight=8, nice=1 weight=16
Expected: nice=0 gets ~2x more CPU than nice=1
nice=0 count: 2506505
nice=1 count: 1256146
Ratio (nice=0 / nice=1): 1.99
Expected: ~2.00

===== CFS Test Complete =====
PASS
PASS ALL TESTS

```

모든 test 다 기대하는 결과값에 근접함을 알 수 있다.

3. testcase3

CPUS=1에서 기댓값 대로 결과가 잘 나옴을 확인할 수 있다.

[CFS Scheduler Test]

TEST: 1. Validate return value of set_nice system call

[EXPECT] allows [-3, 2] and returns 1 on fail.

```
set_nice(-4) -> Result: 1 (Expected: 1) : PASS
set_nice(-3) -> Result: 0 (Expected: 0) : PASS
set_nice(-2) -> Result: 0 (Expected: 0) : PASS
set_nice(-1) -> Result: 0 (Expected: 0) : PASS
set_nice(0)  -> Result: 0 (Expected: 0) : PASS
set_nice(1)  -> Result: 0 (Expected: 0) : PASS
set_nice(2)  -> Result: 0 (Expected: 0) : PASS
set_nice(3)  -> Result: 1 (Expected: 1) : PASS
```

TEST: 2. Wakeup Test (CPUS=1)

```
[EXPECT] After child wakes up, P and C should interleave evenly.
```

PPPPPPPPPPPPPCPCPCPCPCPCPCPCPCPCPC

TEST: 3. Nice value Fork Inheritance (CPUS=1)

```
[EXPECT] P and C should interleave evenly (e.g., PPCPCPCP...).
```

PPCPCPCPCPCPCPCPCPCPCPCPCPCPCPCPC

TEST: 4. Nice value CPU Share (CPUS=1)

```
[EXPECT] H 16 times, L 1 time, H 16 times ...
```

LHHHHHHHHHHHHHHHHHHHLLHHHHHHHHHHHHHHHHHHHLLHHHHHHHHHHHHHHHHHHHLLHHHHHHHHHHHHHHHHLLL

TEST: 5. Check for deadlocks in Multiprocessor environment

[EXPECT] No deadlocks occur & Process ends in A, B, C, D, E, F order or similar.

```
[a: -3] [b: -2] [c: -1] [d: 0] [e: 1] [f: 2]
```

[illegible]

ALL TESTS COMPLETED.

PASS

PASS ALL TESTS

멀티 CPU 환경에서는 deadlock 발생 여부만 확인해보았다.

```
=====
TEST: 5. Check for deadlocks in Multiprocessor environment
=====
[EXPECT] No deadlocks occur & Process ends in A, B, C, D, E, F order or similar.

[a: -3] [b: -2] [c: -1] [d: 0] [e: 1] [f: 2]

abcdfbeabaccdeeadbcfcfbacacbacbadbaadaceccbaebdbbdacbcbadbacdbaccbabacfbfaeabeadcbedcbaadcadbcbaadacacebcebadfadcacdbfbcfca
cbaabdcadbbaeadcbcbacabadacedacdbbbecfacbacbadbfadcdcaabcbceaddbaccebaabdadbbeaacadbbaabcbafdbadbccaedcbbfaffacbbdfbd
aaccbdeeebadbaabcbadbacbaeaccfceaacacbbabcdddcdbdaabebedacacbcdcfcedaabcdbdcbeaaabccddceabdbdcaacbcadacfecabbafeebeacadb
acbaaceabdbdddcbaebcbabadcacbabccabfafeddcbbAbdceecdffcbbdbedbedfcedbeBfdfcdfceecdccfecedCfedefdeffeddfefdedfedffedf
eedfdeffedfedfedfedffeddeffdedfedfedfdeedfefdfdefeDefefefefefefefefefefeeefefefefefefefefeeffeeffEfffffffffffffffffffff
=====

ALL TESTS COMPLETED.
=====
PASS
PASS ALL TESTS
```

deadlock이 일어나지 않음을 확인할 수 있다.

4. testcase4

여러 가지 상황을 확인하는 testcase도 모두 CPUS=1 환경에서 정상적으로 작동함을 확인했다.

```

=====
CFS Scheduler Test Suite
(CPUS=1 mode)
=====

=== Test 1: set_nice boundary ===
[PASS] set_nice(-3) returns 0
[PASS] set_nice(-2) returns 0
[PASS] set_nice(-1) returns 0
[PASS] set_nice(0) returns 0
[PASS] set_nice(1) returns 0
[PASS] set_nice(2) returns 0
[PASS] set_nice(-4) returns 1
[PASS] set_nice(3) returns 1
[PASS] set_nice(-100) returns 1
[PASS] set_nice(100) returns 1

=== Test 2: nice inheritance via fork ===
[PASS] child inherits parent nice and validates correctly
[PASS] child inherits nice=2, rejects nice=3

=== Test 3: initial nice value ===
[PASS] process starts with valid nice=0

=== Test 4: scheduling order by nice value ===
Expected order: nice=-3 finishes first, nice=2 finishes last
Completion order: -3 0 2
[PASS] lower nice finishes before higher nice

=== Test 5: vruntime inheritance (CPUS=1) ===
Completion order: X then Y
[PASS] lower vruntime child (X) finishes before higher vruntime child (Y)

=== Test 6: wakeup vruntime update ===
woken process completed
[PASS] process wakes from sleep and continues execution

=== Test 7: multi-process stability (CPUS=1) ===
[PASS] 6 processes with different nice values all complete

=== Test 8: nice inheritance chain ===
[PASS] nice inheritance works through fork chain

=== Test 9: kill sleeping process ===
[PASS] kill sleeping process does not crash system

=== Test 10: single process basic operation ===
[PASS] single child process runs and exits cleanly

=====
PASS: 20 / FAIL: 0
ALL TESTS PASSED
=====

```

Troubleshooting

1. Lock 관련 순서 및 deadlock 발생의 문제

현재 여러 lock을 쓰고 있기 때문에 multiprocessor 환경에서는 더욱 deadlock 발생할 확률이 크다. 따라서 lock 순서를 미리 $p \rightarrow \text{lock}$ 이 $\text{cfs_lock}(\text{tree insert/delete lock})$ 보다 먼저 사용되어야 하고, $p \rightarrow \text{lock}$ 이 tickslock 보다 먼저 사용되어야 한다고 정했다. 여기서 cfs_lock 과 tickslock 은 겹칠 상황이 없어 우선순위를 정하지 않아도 문제가 될 건 없어보였다. 이렇게 정의하고 코드를 작성하는 것까지는 성공하였으나, 논리적으로 의문점이 드는 부분이 발생했다.

```
void
wakeup(void *chan)
{
    struct proc *p;
    uint64 myVruntime;

    for(p = proc; p < &proc[NPROC]; p++) {
        if(p != myproc()){
            acquire(&p->lock);
            if(p->state == SLEEPING && p->chan == chan) {
                release(&p->lock);

                myVruntime = wakeup_vruntime_setting();

                acquire(&p->lock);
                if(p->state == SLEEPING && p->chan == chan){
                    p->state = RUNNABLE;
                    p->vruntime = myVruntime;
                    p->rbNode->key = p->vruntime;
                    p->rbNode->data = (void *)p;

                    acquire(&cfs_lock);
                    rb_insert(&cfs_runnable_tree, p->rbNode);
                    release(&cfs_lock);
                }
            }
            release(&p->lock);
        }
    }
}
```

해당 wakeup function에서 처음에, $p \rightarrow \text{lock}$ 을 획득하고 그 안에서 $\text{wakeup_vruntime_setting}()$ function을 바로 호출하면, $\text{wakeup_vruntime_setting}()$ 안에 cfs_lock 이 걸리고 또 $p \rightarrow \text{lock}$ 이 걸려 deadlock이 발생할 확률이 커보였다. (kkill에도 위의 코드와 유사하게 설계되어있다)

따라서 wakeup_vruntime_setting을 call할 때는 lock이 걸리지 않는 상황이어야 한다고 판단했다. 먼저 $p \rightarrow \text{lock}$ 을 건 후 $p \rightarrow \text{state} == \text{SLEEPING}$ 인 조건과 $p \rightarrow \text{chan} == \text{chan}$ 인 조건을 만족한다면 다시 $p \rightarrow \text{lock}$ 을 해제하고 myVruntime을 계산한 뒤 다시 $p \rightarrow \text{lock}$ 을 걸어 한번 더 process 조건을 확인해주어야 했다. 이는, 2번의 $p \rightarrow \text{lock}$ 이 걸리는 중간에 process가 wakeup할 조건을 만족하지 않을 수도 있기 때문이었다.

처음에는 이를 모르고 조건을 확인하지 않은 채 process state 및 vruntime 등의 정보만 update 하고 runnable tree에 삽입했으나, 이후 test 시 이런 문제점을 발견하여 2번 조건을 확인하도록 수정하였다.

두번 조건을 check하는 방법 외에 더 좋은 방법이 있을 것 같으나, 현재로써 wakeup에서의 deadlock을 피하면서 vruntime을 설정하는 방법으로 위의 방법을 채택하였다.

2. 멀티 CPU 환경에서의 testcase 결과가 의도한 것에서 벗어난 부분

멀티 CPU 환경에서 deadlock 발생은 하지 않음은 확인해보았지만, CPUS=1처럼 wakeup, fork 등에서 test했을 때 의도치 않은 결과를 냄을 확인하였다. lock 순서도 정확히하고 race condition이 일어날 만한 부분에서 lock으로 보호해주었음에도 명확히 해결되지 않는 문제가 있었다. 따라서 해당 코드에서 멀티 CPU 환경에 대한 추가적인 개선이 필요함을 인지하였다.